

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

PROJECT REPORT

PROCESS TERMINATION MESSAGE SYSTEM

Section: 08

Team: 10

Team Members

2520030432 – K ANUSRI

2520030440 – K ANSHU

2520030193 – K DHEERAJ

2520030028 – B MOHITH

Academic Year: 2026–27 | Term-I

1. Abstract

The Process Termination Message System is a Linux-based C project designed to monitor and report the termination status of child processes. The project demonstrates process creation, parent-child relationships, synchronization, and signal-based termination. The parent process creates multiple child processes using `fork()`. Some children terminate normally using `exit()`, while another child is terminated abnormally using `SIGKILL`. The parent collects the termination status using `wait()` and identifies whether each child terminated normally or due to a signal.

2. Problem Statement

In a multitasking operating system, a parent process may create multiple child processes that can terminate at different times and in different ways. The parent process must properly detect and collect the termination status of its children. This project implements a Linux-based system that demonstrates normal and abnormal child process termination and reports the corresponding process ID, exit status, or terminating signal.

3. Objectives

- Create multiple child processes using `fork()`.
- Monitor and detect child-process termination.
- Use `wait()` to collect and analyse child termination status.
- Identify normal and abnormal termination.
- Display exit status for normally terminated children.
- Display the signal number for signal-terminated children.
- Prevent zombie processes by collecting all child processes.

4. Methodology

The project is implemented in C on Ubuntu/Linux using GCC. The parent process creates three child processes using `fork()`. Child-1 and Child-2 complete their tasks normally and terminate using `exit(10)` and `exit(20)`, respectively. Child-3 continues running in a loop. After a delay, the parent sends `SIGKILL` to Child-3. The parent then repeatedly calls `wait()` to collect each terminated child and uses the standard `wait-status` macros to determine the termination type.

- Parent creates Child-1, Child-2, and Child-3.

- Child-1 terminates normally with exit status 10.
- Child-2 terminates normally with exit status 20.
- Child-3 keeps running until the parent sends SIGKILL.
- The parent collects all children using wait().
- The termination report displays PID, termination type, exit status, or signal number.

5. Linux APIs and Termination Macros Used

API / Macro	Purpose
fork()	Creates a new child process.
wait()	Waits for and collects a terminated child process.
getpid()	Returns the process ID of the calling process.
exit()	Terminates a process normally with an exit status.
kill()	Sends a signal to a process.
SIGKILL	Forces abnormal termination of the selected child.
WIFEXITED()	Checks whether the child terminated normally.
WEXITSTATUS()	Obtains the exit status of a normally terminated child.
WIFSIGNALED()	Checks whether the child terminated because of a signal.
WTERMSIG()	Obtains the signal number that caused termination.

6. Source Code

The following screenshots show the complete C source code entered and saved in the Ubuntu terminal using GNU Nano.

```
GNU nano 8.7.1                                termination.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

int main()
{
    pid_t child1, child2, child3;
    pid_t terminated_pid;
    int status;

    printf("=====\n");
    printf("PROCESS TERMINATION MESSAGE SYSTEM\n");
    printf("Parent PID : %d\n", getpid());
    printf("=====\n\n");

    child1 = fork();

    if(child1 == 0)
    {
        printf("[Child-1] PID=%d started\n", getpid());
        sleep(2);
        printf("[Child-1] Completing task normally\n");
        exit(10);
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^M-U Undo
    ^X Exit      ^R Read File  ^N Replace    ^U Paste      ^_ Justify    ^_/ Go To Line  ^M-E Redo
```

```
GNU nano 8.7.1                                termination.c
    child2 = fork();

    if(child2 == 0)
    {
        printf("[Child-2] PID=%d started\n", getpid());
        sleep(4);
        printf("[Child-2] Completing task normally\n");
        exit(20);
    }

    child3 = fork();

    if(child3 == 0)
    {
        printf("[Child-3] PID=%d started\n", getpid());

        while(1)
        {
            printf("[Child-3] Running...\n");
            sleep(1);
        }
    }

    sleep(3);

    printf("\nParent sending SIGKILL to Child-3 (PID=%d)\n\n", child3);

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^M-U Undo
    ^X Exit      ^R Read File  ^N Replace    ^U Paste      ^_ Justify    ^_/ Go To Line  ^M-E Redo
```

```

GNU nano 8.7.1                                termination.c

kill(child3, SIGKILL);

printf("===== TERMINATION REPORT =====\n");
while((terminated_pid = wait(&status)) > 0)
{
    printf("\nChild PID : %d\n", terminated_pid);
    if(WIFEXITED(status))
    {
        printf("Termination Type : NORMAL\n");
        printf("Exit Status : %d\n", WEXITSTATUS(status));
    }
    else if(WIFSIGNALED(status))
    {
        printf("Termination Type : ABNORMAL\n");
        printf("Killed By Signal : %d\n", WTERMSIG(status));
    }
}

printf("\n===== \n");
printf("All child processes collected.\n");
printf("No zombie processes exist.\n");
printf("Project Execution Completed.\n");
printf("===== \n");

return 0;
}

```

7. Source Code Creation and Saving

The source file termination.c was created in the Ubuntu home directory using GNU Nano. The code was pasted into the editor and saved using Ctrl+O followed by Enter, then Nano was closed using Ctrl+X.

```

k_anusri@DESKTOP-TM20BAQ:~$ cd ~/OneDrive/Documents/OSSP
-bash: cd: /home/k_anusri/OneDrive/Documents/OSSP: No such file or directory
k_anusri@DESKTOP-TM20BAQ:~$ ls
OSSP_Skill      exp2_filecopy.c      exp6_fifo          exp8_memory.c      ossp_fifo
OS_Week2       exp3_process_states  exp6_fifo.c        exp9_file_operations output.txt
command_exec.c exp3_process_states.c exp7_memory         exp9_file_operations.c process
command_exec.c.save.1 exp4_multiple_children exp7_memory.c       experiment2         process.c
exp1_process    exp4_multiple_children exp8_cow            experiment2.c       redirected_output.txt
exp1_process.c  exp5_pipe            exp8_cow.c          input.txt           stdlib_output.txt
exp2_filecopy   exp5_pipe.c          exp8_memory         os_command.c
k_anusri@DESKTOP-TM20BAQ:~$ find - -name "termination.c" 2>/dev/null
k_anusri@DESKTOP-TM20BAQ:~$ nano termination.c
k_anusri@DESKTOP-TM20BAQ:~$ ls -l termination.c
-rw-r--r-- 1 k_anusri k_anusri 1038 Sep 24 03:26 termination.c

```

8. Compilation and Execution

The program was compiled with GCC using the command below and then executed from the Ubuntu terminal.

```
gcc termination.c -o termination
./termination
```

```

k_anusri@DESKTOP-TM20BAQ:~$ gcc termination.c -o termination
k_anusri@DESKTOP-TM20BAQ:~$ ./termination
=====
PROCESS TERMINATION MESSAGE SYSTEM
Parent PID : 543
=====

[Child-1] PID=544 started
[Child-2] PID=545 started
[Child-3] PID=546 started
[Child-3] Running...
[Child-3] Running...
[Child-1] Completing task normally
[Child-3] Running...

Parent sending SIGKILL to Child-3 (PID=546)

===== TERMINATION REPORT =====

Child PID : 544
Termination Type : NORMAL
Exit Status : 10

Child PID : 546
Termination Type : ABNORMAL
Killed By Signal : 9
[Child-2] Completing task normally

Child PID : 545
Termination Type : NORMAL
Exit Status : 20

=====
All child processes collected.
No zombie processes exist.
Project Execution Completed.
=====

```

9. Output Analysis

- The parent process displays its PID and creates three child processes.
- Child-1 completes normally and reports Exit Status 10.
- Child-2 completes normally and reports Exit Status 20.
- Child-3 is terminated by SIGKILL, which is signal number 9.
- WIFEXITED() and WEXITSTATUS() identify normal termination and exit codes.
- WIFSIGNALED() and WTERMSIG() identify signal-based abnormal termination.

- The final message confirms that all child processes were collected and no zombie processes remain.

10. Execution Flow

- Start the parent process.
- Create Child-1 using fork().
- Create Child-2 using fork().
- Create Child-3 using fork().
- Child-1 exits normally with status 10.
- Child-2 exits normally with status 20.
- Child-3 continues running.
- Parent sends SIGKILL to Child-3.
- Parent calls wait() and collects each terminated child.
- Termination information is displayed.
- All child processes are collected and the program completes.

11. Result

The Process Termination Message System was successfully implemented and executed on Ubuntu/Linux. The program demonstrated both normal and abnormal process termination. The parent correctly collected the termination status of all three child processes and displayed the corresponding exit status or signal number.

12. Conclusion

The project demonstrates important Linux process-management concepts including process creation with fork(), normal termination with exit(), signal-based termination using SIGKILL, and child-status collection using wait(). The wait-status macros provide a clear way to distinguish normal and abnormal termination. Properly collecting all children also prevents zombie processes.

13. Team Contributions

Member	Contribution
K ANUSRI (2520030432)	fork() implementation and overall integration
K ANSHU (2520030440)	Normal child-process termination

K DHEERAJ (2520030193)	Abnormal termination and signal handling
B MOHITH (2520030028)	Testing, output analysis, documentation and GitHub

14. Tools and Technologies

- Ubuntu/Linux
- C Programming Language
- GCC Compiler
- GNU Nano / Text Editor
- Linux Terminal
- GDB for debugging (where required)
- GitHub for project version control

15. GitHub Repository

Repository: https://github.com/kanusri-tmj/Jarvis-kernel_OSSP

16. Final Verification

- Source file: termination.c
- Compilation completed without errors.
- Program executed successfully.
- Three child processes were created.
- Normal termination was detected for Child-1 and Child-2.
- Abnormal signal termination was detected for Child-3.
- Exit statuses 10 and 20 were reported.
- Signal 9 (SIGKILL) was reported.
- All child processes were collected.
- No zombie processes were reported.